

# TÀI LIỆU PHẢN BIỆN KỸ THUẬT SONG NGỮ BILINGUAL TECHNICAL DEFENSE DOCUMENT

## CRT Sparse Sort (Thanh Ha Algorithm)

Author: Hao T. Nguyen (Nguyễn Thanh Hà)

### CÂU 1 / QUESTION 1

#### [VIE] Về tính mới và bản chất thuật toán so với Radix Sort truyền thống

**Câu hỏi:** Bản chất thuật toán là băm số dư (Modulo) rồi dùng Radix Sort sắp xếp các trục tọa độ. Thuật toán này khác gì một bản sao của Radix Sort truyền thống (cơ số 10 hoặc nhị phân) để được coi là một đóng góp mới?

**Trả lời:** Thuật toán chia sẻ nguyên lý phi so sánh tuyến tính của Radix Sort nhưng sở hữu 3 điểm khác biệt cốt lõi về bản chất hình học và vi kiến trúc:

- Không chế kích thước mảng bộ đếm (Modulo Compression):** Radix Sort truyền thống cố định cơ số (ví dụ cơ số 256) khiến mảng đếm phân tán trên RAM khi dải số lớn. Thuật toán này ép không gian dữ liệu về 9 trục số dư cực hẹp ( $p_{\max} = 29$ ). Mảng bộ đếm luôn cố định từ 3 đến 29 phần tử, đảm bảo nằm trọn 100% trong L1/L2 Cache của CPU, triệt tiêu hiện tượng Cache Misses.
- Tận dụng tính chất đối xứng gương (Mirroring Optimization):** Mô hình hóa cấu trúc trên siêu hình xuyên (Torus) giúp xác định tâm đối xứng gương  $M_{\text{MID}}$ . Thuật toán chỉ cần tính toán tọa độ cho 50% phần tử ở nửa dưới; nửa trên được giải gương tự nhiên bằng cách quét ngược bộ nhớ với chi phí  $\mathcal{O}(0)$  đảo mảng.
- Tính độc lập đại số tuyệt đối (Parallel Independence):** Các chữ số của Radix Sort truyền thống phụ thuộc nhau theo thứ tự vị trí (LSB/MSB). Ngược lại, 9 trục số dư CRT hoàn toàn độc lập đại số, cho phép tính toán song song đồng thời trên 9 ALUs riêng biệt của phần cứng (GPU/FPGA).

#### [ENG] On Novelty and Core Mechanism Compared to Traditional Radix Sort

**Question:** The algorithm essentially hashes residues (Modulo) and uses Radix Sort to sort along coordinate axes. What makes this different from traditional Radix Sort (base-10 or binary) to be considered a new contribution?

**Answer:** While sharing the linear non-comparative sorting principle with Radix Sort, this algorithm possesses three core architectural and microarchitectural differences:

- Modulo Compression (Counter Array Constraint):** Traditional Radix Sort uses fixed bucket sizes (e.g., base-256), causing counter arrays to scatter across RAM when data ranges expand. This algorithm compresses a range of 2 billion numbers into 9 ultra-narrow residue axes ( $p_{\max} = 29$ ). The counter array remains fixed between 3 and 29 elements, guaranteeing 100% L1/L2 Cache residency and eliminating Cache Misses.
- Mirroring Optimization:** Modeling the structure on a higher-dimensional torus reveals a mirror symmetry midpoint ( $M_{\text{MID}}$ ). The algorithm computes coordinates for only 50% of elements in the lower half; the upper half is un-mirrored during reverse memory traversal with  $\mathcal{O}(0)$  array reversal overhead.
- Parallel Independence:** Digits in traditional Radix Sort depend on positional significance (LSB/MSB). Conversely, the 9 CRT residue axes are algebraically independent, enabling full parallel processing across 9 separate hardware ALUs (GPU/FPGA).

## CÂU 2 / QUESTION 2

---

### [VIE] Về thắt nút cổ chai phần cứng khi thực hiện phép toán lấy dư (%)

**Câu hỏi:** Thuật toán đạt độ phức tạp tuyến tính  $\mathcal{O}(K \cdot N)$  nhưng bước phân rã (decompose) bắt CPU gánh 9 phép toán lấy dư (%) cho mỗi phần tử. Phép toán % hay / trên CPU tốn từ 10–40 chu kỳ máy, nặng hơn phép so sánh 1 chu kỳ của QuickSort. Làm sao giải quyết điểm nghẽn này?

**Trả lời:** Trong kiến trúc máy tính hiện đại, điểm nghẽn lớn nhất không nằm ở tính toán số học (Arithmetic), mà nằm ở độ trễ truy cập bộ nhớ (Memory Wall) và rẽ nhánh sai (Branch Misprediction) do các câu lệnh so sánh `if`:

1. **Xóa bỏ lệnh rẽ nhánh:** `std::sort` (Introsort) liên tục so sánh ngẫu nhiên làm bộ dự đoán nhánh (Branch Predictor) bị sai, gây trống hàng đợi pipeline. CRT Sparse Sort xử lý dòng dữ liệu phẳng hoàn toàn không có lệnh rẽ nhánh.
2. **Tối ưu hóa thời gian biên dịch (Compile-time Magic Numbers):** Các số nguyên tố  $p_i$  được thiết kế dưới dạng hằng số cố định. Ở chế độ biên dịch tối ưu (`-O3`), trình biên dịch tự động chuyển toàn bộ phép chia/lấy dư thành phép nhân hằng số ma thuật (Magic Numbers) kết hợp dịch bit. Thao tác này chỉ tốn từ 1–2 chu kỳ máy, tương đương phép cộng.
3. **Tối ưu Cache Locality:** Dữ liệu được đóng gói gọn trong khóa 64-bit (`packed_key`), duy trì tính định vị không gian và thời gian tối đa trên bộ nhớ đệm.

### [ENG] On Hardware CPU Bottleneck During Modulo Operations (%)

**Question:** The algorithm claims linear complexity  $\mathcal{O}(K \cdot N)$ , but the decomposition phase forces the CPU to execute 9 modulo operations (%) per element. Modulo operations on modern CPUs cost 10–40 machine cycles, far heavier than a 1-cycle comparison in QuickSort. How is this bottleneck resolved?

**Answer:** In modern computer architectures, the primary bottleneck is not arithmetic computation, but Memory Latency (Memory Wall) and Branch Mispredictions from conditional `if` statements:

1. **Branch-Free Execution:** `std::sort` (Introsort) performs continuous random comparisons, causing frequent Branch Predictor misses that empty the pipeline queue. CRT Sparse Sort processes flat data streams with zero conditional branches.
2. **Compile-Time Magic Numbers:** Prime moduli  $p_i$  are fixed compile-time constants. Under release compiler optimizations (`-O3`), compilers automatically convert all 9 modulo/division operations into magic number multiplications and bit shifts. This reduces operation costs to 1–2 machine cycles (equivalent to addition), eliminating division overhead.
3. **Cache Locality:** Key data is tightly packed into a 64-bit integer (`packed_key`), maintaining optimal spatial and temporal cache locality.

## CÂU 3 / QUESTION 3

---

### [VIE] Về xử lý mật độ dữ liệu siêu thưa (Sparse Data)

**Câu hỏi:** Không gian tạo bởi 9 số nguyên tố lên tới hơn 3.2 tỷ điểm. Khi đưa 1.000 số ngẫu nhiên vào không gian này, mật độ trống lên tới 99.99%. Làm sao thuật toán tránh khỏi việc lãng phí thời gian duyệt qua hàng tỷ phân đoạn trống?

**Trả lời:** Thuật toán dựa trên cấu trúc mảng phẳng liên tục (`std::vector`) thay vì khởi tạo ma trận không gian thực tế:

1. Thuật toán không bao giờ khởi tạo hay duyệt qua 3.2 tỷ đoạn trống. Dữ liệu  $N$  phần tử ban đầu được giữ nguyên trong mảng phẳng kích thước đúng bằng  $N$ .
2. Hàm sắp xếp phân phối các phần tử dựa trên bộ số dư thông qua mảng đếm tích lũy (Prefix Sum).
3. Số lượng bước lặp phụ thuộc duy nhất vào số lượng phần tử thực tế ( $N$ ) nhân với số trục ( $K$ ), tức  $\mathcal{O}(K \cdot N)$ . Toàn bộ không gian thừa đã bị nén bẹp về mặt hình học, không tốn thêm chu kỳ máy hay dung lượng RAM nào cho khoảng trống.

### [ENG] On Handling Hyper-Sparse Data Density

**Question:** The coordinate space generated by 9 prime numbers spans over 3.2 billion points. When placing 1,000 random integers into this space, empty density exceeds 99.99%. How does the algorithm avoid wasting time iterating over billions of empty slots?

**Answer:** The algorithm relies on a continuous flat array (`std::vector`) rather than instantiating an actual spatial matrix:

1. The algorithm never allocates or iterates through 3.2 billion empty slots. Input data of  $N$  elements remains stored in a flat vector of exact size  $N$ .
2. The sorting function distributes elements based on residue vectors using prefix sums.
3. Iteration steps depend strictly on the actual element count ( $N$ ) multiplied by the axis count ( $K$ ), yielding  $\mathcal{O}(K \cdot N)$  complexity. The entire sparse space is geometrically collapsed, consuming zero extra CPU cycles or memory.

## CÂU 4 / QUESTION 4

---

### [VIE] Về độ ổn định và trường hợp xấu nhất (Worst-case)

**Câu hỏi:** Nếu dải số đầu vào tăng từ 2 tỷ lên 100 tỷ hay 1000 tỷ, thuật toán bắt buộc phải nạp thêm các số nguyên tố lớn hơn ( $K$  tăng lên 13–15). Khi  $K$  tăng, thuật toán có còn giữ được lợi thế so với `std::sort`?

**Trả lời:** Độ ổn định của CRT Sparse Sort vượt trội ở các điểm sau:

1. **Tốc độ tăng của  $K$  cực chậm:** Do tích các số nguyên tố (Primorial  $\prod p_i$ ) tăng theo hàm mũ, số lượng trục  $K$  chỉ tăng theo hàm logarit của dải số. Để mở rộng không gian lên 1000 tỷ, chỉ cần bổ sung thêm khoảng 4–5 số nguyên tố.
2. **Không suy biến độ phức tạp:** Trường hợp xấu nhất (Worst-case) của CRT Sparse Sort luôn bằng trường hợp tốt nhất (Best-case), cố định ở mức  $\mathcal{O}(K \cdot N)$ . Thuật toán hoàn toàn không bị suy biến về  $\mathcal{O}(N^2)$  như QuickSort khi gặp mảng dữ liệu có cấu trúc đặc biệt (mảng đã sắp xếp hoặc mảng ngược).

### [ENG] On Stability and Worst-Case Performance

**Question:** If the input range increases from 2 billion to 100 billion or 1 trillion, larger prime numbers must be added ( $K$  increases to 13–15). As  $K$  grows, does the algorithm maintain its advantage over `std::sort`?

**Answer:** CRT Sparse Sort maintains superior stability due to the following characteristics:

1. **Logarithmic Growth of  $K$ :** Because the product of prime numbers (Primorial  $\prod p_i$ ) grows exponentially, the required number of axes  $K$  grows logarithmically relative to the data range. Expanding to 1 trillion requires adding only 4–5 primes.
2. **Non-Degenerate Complexity:** The worst-case complexity of CRT Sparse Sort strictly equals its best-case complexity at  $\mathcal{O}(K \cdot N)$ . Unlike QuickSort, it never degenerates to  $\mathcal{O}(N^2)$  when encountering structured patterns (e.g., pre-sorted or reverse-sorted arrays).

## CÂU 5 / QUESTION 5

---

### [VIE] Về tính ứng dụng thực tiễn và định vị công trình

**Câu hỏi:** Tại sao các thư viện chuẩn hiện nay vẫn dùng `std::sort` hay Introsort làm mặc định mà không dùng mô hình số dư này? Định hướng áp dụng thực tế của công trình là gì?

**Trả lời:** Sự khác biệt nằm ở bản chất đa dụng (General-purpose) và chuyên dụng (Special-purpose):

1. `std::sort` là thuật toán đa dụng, phải xử lý được mọi kiểu dữ liệu từ số thực, chuỗi ký tự đến các struct phức tạp thông qua toán tử so sánh  $<$ .
2. CRT Sparse Sort dựa trên toán học đồng dư, là thuật toán chuyên dụng (Special-purpose) tối ưu riêng cho dữ liệu số nguyên.
3. **Định hướng ứng dụng cốt lõi:**
  - *Hệ thống nhúng & Chip xử lý tín hiệu số (DSP):* Nơi phần cứng vốn đã giao tiếp bằng Hệ Thống Số Dư (RNS), thuật toán giúp sắp xếp dữ liệu trực tiếp mà không tốn chi phí chuyển đổi ngược về hệ thập phân.
  - *Bộ tăng tốc phần cứng song song (GPU / FPGA):* Tận dụng kiến trúc Pipeline và hàng ngàn lõi xử lý nhỏ để tính toán 9 trục đồng thời, đưa thời gian phân rã mảng thừa về tiệm cận  $\mathcal{O}(1)$  trên phần cứng.

### [ENG] On Practical Application and Positioning

**Question:** Why do standard libraries still use `std::sort` or Introsort by default instead of residue models? What is the practical application roadmap for this work?

**Answer:** The difference lies in general-purpose vs. special-purpose algorithm design:

1. `std::sort` is a general-purpose algorithm designed to handle arbitrary data types (floats, strings, custom structs) via comparison operators ( $<$ ).
2. CRT Sparse Sort is based on residue arithmetic, making it a domain-specific algorithm optimized specifically for integer datasets.
3. **Target Application Roadmap:**
  - *Embedded Systems & Digital Signal Processors (DSPs):* In Residue Number System (RNS) hardware pipelines, this algorithm allows direct sorting of native RNS data without back-conversion to positional notation.
  - *Parallel Hardware Accelerators (GPU / FPGA):* Leveraging pipelining and thousands of compute cores allows concurrent execution across all 9 axes, bringing sparse sorting time near  $\mathcal{O}(1)$  on dedicated hardware.